Campus Incident Reporting App — Technical Report

Nov 10, 2026 · @Darlene

1. Introduction and Problem Statement

The Campus Incident Reporting App is a Flutter and Firebase mobile app that gives ALU students one channel to report campus problems and track them until staff resolve them.

Today, issues such as a broken tap, a power cut in a residence or an unsafe stairwell are reported informally: by word of mouth, in WhatsApp groups or at a front desk. Reports get lost, students cannot see whether anything is being done, and staff have no single list to prioritise from. The same problem is often reported many times, while other problems are never reported because students assume nobody will act.

Author: [Your name] · **Course:** [Course name and code] · **Repository:** [GitHub link]

2. Objectives, Scope and Users

The app lets a student file a structured report in under a minute, and lets staff move every report through a three-step lifecycle: **Submitted → In Progress → Resolved**.

Objectives

1. Restrict the app to the ALU community through institutional email domains.
2. Capture complete reports: category, title, description, campus location and an optional photo.
3. Let anyone browse, search and filter campus reports, and see each report's time-stamped status history.
4. Let staff update statuses, with every change appearing live on the reporter's screen.
5. Protect privacy and resist misuse through anonymous reporting and server-side security rules.

Users and roles

Role	How the account is created	What they can do
Student	Registers with `@alustudent.com` or `@alueducation.com`, then verifies the email	Report, browse, track, edit or delete own report while *Submitted*

Role	How the account is created	What they can do
Staff	Registers with `@aluedu.com`; created as staff with no verification step	Everything students can, plus change any report's status

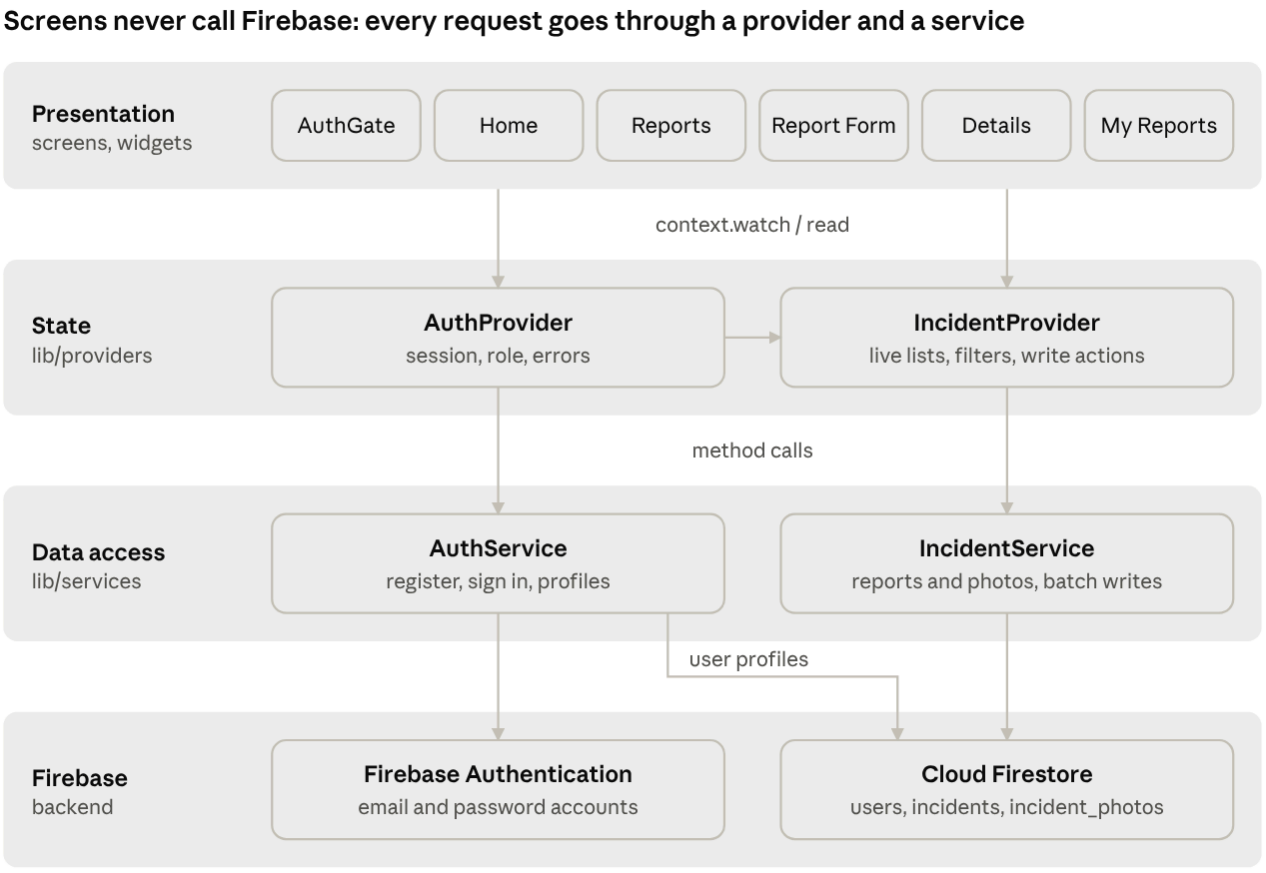
Campus-specific design decisions

Decision	Reason
Five categories: Facilities, Water, Electricity, Safety, Sanitation	They match the teams that handle campus issues. Damaged furniture and broken equipment go under Facilities to keep the list short.
Fixed list of campus locations	Free text produced spellings staff could not group ("lib", "Library"). A dropdown keeps the data consistent.
Three statuses only	Matches how facilities teams work and keeps the student's timeline easy to read.
Anonymous reporting	Safety and sanitation reports can be sensitive; students should not fear being identified.
Edit or delete only while *Submitted*	Students can correct mistakes but cannot rewrite or remove a report once staff have started on it.

Out of scope: push notifications, an admin dashboard, and analytics.

3. System Architecture

The app uses a four-layer architecture in which each layer only talks to the layer directly below it, so screens never import Firebase.



Layered architecture · 4 layers

The small arrow between the two providers is the `ChangeNotifierProxyProvider`, which hands the signed-in user to `IncidentProvider`. `AuthService` also writes and reads the user's profile in Firestore.

Layer	Folder	Responsibility
Presentation	`lib/screens`, `lib/core/widgets`	Layout, forms, navigation, reusable widgets
State	`lib/providers`	App state, business rules (who may edit, delete or update), friendly error messages
Data access	`lib/services`	The only code that calls Firebase; replaced by fakes in tests
Models	`lib/models`	Typed `Incident`, `StatusChange`, `AppUser` with `fromMap` / `toMap`
Core	`lib/core`	Constants (categories, statuses, locations, email domains), theme, validators

Tech stack: Flutter 3.47 (Dart 3), `firebase_auth`, `cloud_firestore`, `provider`, `image_picker`, `google_fonts`, `intl`.

4. State Management

The app uses **Provider with `ChangeNotifier`**: two providers hold all shared state, and Firestore streams push changes into them so every screen updates without manual refreshes.

- **`AuthProvider`** listens to `FirebaseAuth.authStateChanges()` and exposes `isSignedIn`, `isEmailVerified`, `currentUser`, `isStaff`, `isSubmitting` and `errorMessage`. `AuthGate` watches it and picks the screen (login, verify email, or the app), so sign-in, sign-out and session restore after a restart need no navigation code.
- **`IncidentProvider`** holds two live streams (all reports, and the user's own reports), the search and filter state, the photo cache, and every write action (submit, edit, update status, delete).
- A **`ChangeNotifierProxyProvider<AuthProvider, IncidentProvider>`** passes the signed-in user into `IncidentProvider`. When the user changes, the old streams are cancelled and new ones opened, so one user's data never leaks into the next session.
- The details screen uses a **`StreamBuilder`** on the single report document. When staff change the status, a student viewing that report sees the timeline move immediately.

Business rules live in the providers, not the widgets. For example, `canModify()` decides who may edit or delete, and `updateStatus()` re-checks the staff role before writing. Screens only read state and call methods.

Why Provider over the alternatives

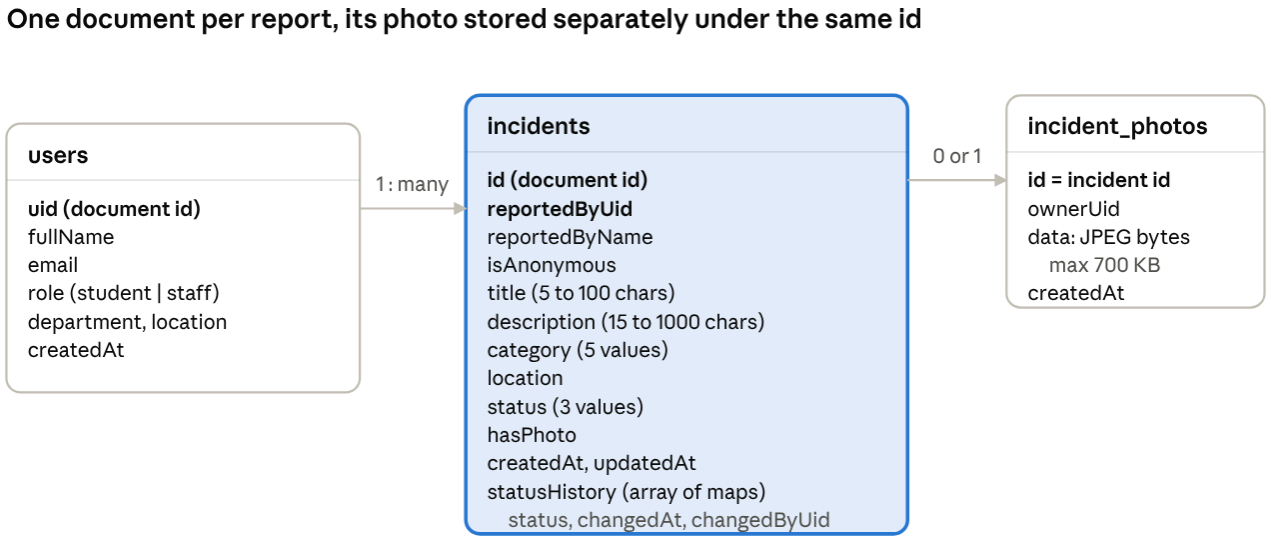
Option	Verdict for this app
`setState` only	Cannot share report lists across Home, Reports, Profile and Details without passing data down manually.
Provider (chosen)	Recommended by the Flutter team for apps of this size, little boilerplate, and Firestore snapshots map directly to `notifyListeners()`.
BLoC / Cubit	Works well, but needs events, states and extra classes for each flow, which adds code without adding value at this scale.

Option	Verdict for this app
Riverpod	Strong option; Provider was chosen because it is simpler to set up and explain for a project of this size.

Services are injected through the provider constructors, which is what makes the providers testable with fakes (section 9).

5. Data Model and Firebase Structure

All data lives in three Cloud Firestore collections; Firebase Authentication holds only the email and password.



Firestore data model · 3 collections

A user has many reports, linked by `reportedByUid`. A report has at most one photo, stored under the same document id.

- **`users/{uid}`**: one profile per account. Using the Auth uid as the document id lets the rules look up a role with a single `get()`.
- **`incidents/{id}`**: one document per report. `statusHistory` is embedded as an array because it is small (at most a few entries), always read with the report, and staff can append to it with `FieldValue.arrayUnion` in the same write as the new `status`.
- **`incident_photos/{id}`**: the optional photo as compressed JPEG bytes (a Firestore `Blob`).

Why photos are in Firestore, not Cloud Storage. New Firebase projects can only use Cloud Storage on the paid Blaze plan. To stay on the free Spark plan:

1. The photo is resized to at most 1024 px at 70% quality on the device, giving roughly 100–300 KB. The app and the rules both reject anything over 700 KB, under Firestore's 1 MiB document limit.
2. Photos live in their **own collection**, so loading report lists never downloads every photo. Each photo is fetched only when shown, then cached for the session.
3. The report and its photo are written in **one atomic batch**, so a report never points at a photo that failed to save. Delete removes both in one batch too.

Other design trade-offs

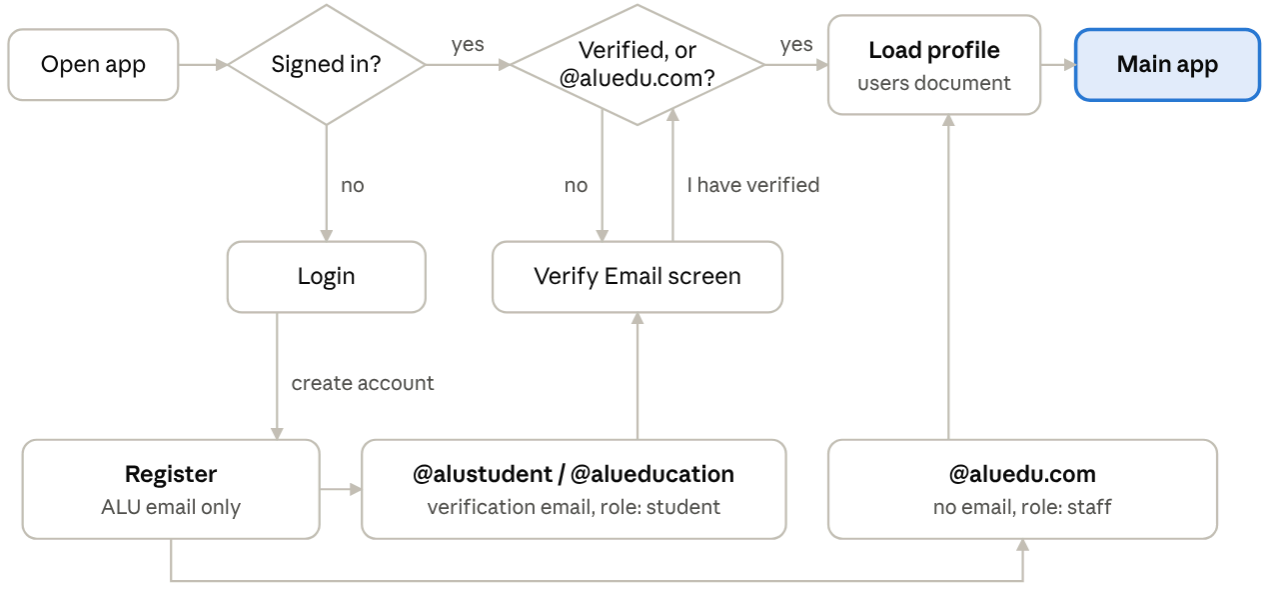
- `reportedByName` is copied onto each report, so lists need no extra read per row and users can still only read their own profile.
- `createdAt` and `updatedAt` use server timestamps, so a device with a wrong clock cannot fake when a report was made.
- The "my reports" query filters only on `reportedByUid` and sorts in Dart, which avoids needing a composite index.

[Insert screenshot: Firebase Console → Firestore showing the three collections, with one incident document expanded.]

6. Authentication Flow

Only ALU email addresses can use the app: student domains must verify their email first, while `@aluedu.com` staff accounts go straight in.

Student domains must verify before entering; @aluedu.com staff go straight in



Authentication flow · AuthGate decisions

`AuthGate` re-runs these decisions every time Firebase reports an auth change, so signing in, signing out (from My Reports) and reopening the app all land on the right screen without navigation code. The session is kept by Firebase between app launches.

The ALU-only rule is enforced at three levels

1. **Form validators** (`validateAluEmail`) give instant inline feedback.
2. **`AuthProvider`** rejects non-ALU emails before any Firebase call, including for password resets.
3. **Security rules** (`hasAluEmail()` in `firestore.rules`) check the email in the user's token, so even a direct API call from a non-ALU account cannot read or write data.

Other session details

- **Forgot password** sends a Firebase reset email, again for ALU addresses only.
- **Registration timing:** Firebase signs a new account in before its profile document is written. `AuthProvider` waits for the write to finish before loading the profile, so new staff never see a "profile not found" error.
- **Staff domain trade-off:** because `@aluedu.com` accounts skip verification, Firebase cannot prove the person owns the address. This keeps staff onboarding fast; section 11 lists stronger alternatives.

[Insert screenshot: Firebase Console → Authentication → Users, showing student and staff accounts.]

7. Security and Role-Based Access Control

Firestore Security Rules (`firestore.rules`) are the real enforcement: the app hides controls a role cannot use, but the server rejects the same actions even if someone calls the Firestore API directly.

Action	Student	Staff	Rule
Read reports and photos	Verified ALU account	Yes	`allow read: if isVerified()`
Create a report	As themselves, status must be *Submitted*, content validated	Yes	`allow create` + `hasValidContent()`

Action	Student	Staff	Rule
Add a photo	Only with their own new report, at most 700 KB	Yes	`incident_photos` create, checked with `getAfter()`
Edit own report	Only while *Submitted*; only title, description, category, location	Not applicable	owner `allow update` with `affectedKeys().hasOnly(...)`
Change status	No	Only `status`, `updatedAt`, `statusHistory`	staff `allow update`
Delete a report	Own report, only while *Submitted*	No	`allow delete`
Read profiles	Own only	Own only	`users/{uid}` read
Set own role	Student domains can only create `student` profiles	Only `@aluedu.com` can create `staff` profiles	`users` create; role is immutable on update

`hasValidContent()` checks that the title is 5–100 characters, the description 15–1000, the category one of the five allowed values, and the location non-empty. The form enforces the same limits, so users see the error before any request is sent.

Privacy

- **Anonymous reports:** the reporter's name is replaced with "Anonymous" *before* it is saved, so nobody can read it from the database, staff included. The user ID is still stored so the ownership rules work and serious abuse can be traced by an administrator.
- **Minimal exposure:** users can only read their own profile document. Other users' emails never reach the client; lists show only the reporter's display name.

Misuse prevention

- Only ALU email domains, and student accounts must verify their email.
- Server-side length limits and a fixed category list stop empty or junk reports.
- Server timestamps (`FieldValue.serverTimestamp()`) stop a device from faking when a report was made.
- Students cannot change statuses, change their role, or rewrite a report once staff have started on it.

8. Features and UI/UX Design

Every core workflow runs end to end against Firestore: submit, browse and filter, view details, track status, edit, delete, and staff status updates.

Screen	What it does	Screenshot
Login / Register	ALU email validation, password toggle, forgot password, inline errors	[Insert screenshot]
Verify Email	"I have verified", resend email, switch account	[Insert screenshot]
Home	Greeting, report button, counters (total, in progress, resolved), 3 most recent reports	[Insert screenshot]
Report Form	Category tiles, title, description, location dropdown, optional photo from camera or gallery, anonymous switch	[Insert screenshot]
Reports	All reports with live search, status chips and category chips	[Insert screenshot]
Incident Details	Summary, status timeline with timestamps, description, photo; staff see "Update Status"	[Insert screenshot of student view and staff view]
My Reports	Profile card, personal counters, status filter, own reports	[Insert screenshot]

Design decisions

- **Reporting is one tap away.** A raised "+" button in the bottom bar opens the form from any tab, because reporting is the main action.
- **Consistent status colours.** Submitted, In Progress and Resolved use the same colours on chips, counters and the timeline, so status reads at a glance.
- **Feedback on every async action.** Buttons show spinners and block double taps; lists have loading, empty and error states with a Retry button; success and failure show snackbars.
- **Forms keep their data on failure.** A lost connection does not force the user to retype a report.
- **Confirmation before destructive actions.** Delete and sign-out each ask for confirmation.

- **Reusable components.** `IncidentCard`, `StatusChip`, `FilterChipRow`, `CategoryTile`, `SummaryTile`, `AppTextField` and `PrimaryButton` are shared, so all screens look the same and a style change happens in one place.
- **Responsive layout.** A responsive shell keeps a phone-width layout centred on tablets and the web; tests check there is no overflow at 360, 390 and 1200 px.

9. Testing

All **102 automated app tests and 48 security-rule tests pass** (`flutter test`), and `flutter analyze` reports no issues.

Approach. Services are injected into the providers, so tests swap in `FakeIncidentService` and `FakeAuthService`, driven by `StreamController`s. This runs the real provider logic and real widgets with no network or Firebase project.

Security rules tests. `rules_test/rules.test.mjs` runs 48 checks of `firestore.rules` against the local Firestore emulator using `@firebase/rules-unit-testing`. They cover the ALU domain check (including a lookalike `aluedu.com.evil.com`), email verification, students being unable to make themselves staff, report validation, owner-only edit and delete while *Submitted*, staff-only status changes, the 700 KB photo limit, and atomic report + photo batches. Run them with `cd rules_test && npm test`.

Test file	What it covers
`models_test.dart`	`toMap` / `fromMap` round trips, defaults for missing fields, unknown enum values
`incident_provider_test.dart`	Live streams, search and filters, counters, submit with and without a photo, oversized photo rejected, photo caching, staff-only status updates, edit and delete ownership rules, anonymous reports
`auth_test.dart`	ALU and staff domain checks, form validators, Firebase error messages, provider rejects non-ALU emails
`report_form_test.dart`, `report_edit_test.dart`	Camera or gallery choice, required fields, minimum lengths, successful submit, data kept on failure, double-tap protection, edit mode pre-fills and saves
`reports_screen_test.dart`	Search, status and category filters, empty, loading and error states

Test file	What it covers
`incident_details_test.dart`	Timeline, staff-only controls, delete flow
`home_screen_test.dart`, `profile_screen_test.dart`	Counters, recent reports, navigation, sign-out confirmation
`layout_test.dart`, `widget_test.dart`	No overflow at 360, 390 and 1200 px; shared widgets

Manual testing on a device

Scenario	Expected result	Result
Register with a Gmail address	Rejected with "Only ALU emails are allowed"	[Pass/Fail]
Register with `@alustudent.com`	Verification email sent; app opens only after verifying	[Pass/Fail]
Register with `@aluedu.com`	Goes straight into the app as staff	[Pass/Fail]
Submit a report with a photo	Appears in Reports and My Reports; photo shows in Details	[Pass/Fail]
Staff changes status on phone A	Student's Details screen on phone B updates without refreshing	[Pass/Fail]
Student tries to edit after status is *In Progress*	Edit and delete options are hidden	[Pass/Fail]
Turn off the internet and submit	Friendly error; form keeps the typed data	[Pass/Fail]

10. Challenges and Solutions

Challenge	Solution
Firebase Storage needs the paid Blaze plan on new projects, so photo upload could not be deployed	Photos are compressed on the device (max 1024 px, 70% quality) and stored as bytes in a separate `incident_photos` Firestore collection, written in the same atomic batch as the report

Challenge	Solution
Writing security rules for two roles without letting students promote themselves	Role is fixed at creation: student domains can only create `student` profiles, only `@aluedu.com` can create `staff`, and the role can never be changed by its owner
Firebase signs a new account in before its profile document exists, so new staff briefly saw "profile not found"	`AuthProvider` waits until registration has saved the profile, then loads it
Keeping Home, Reports, My Reports and Details in sync after a status change	Live Firestore streams in one shared provider, plus a `StreamBuilder` on the details screen
Picking photos on both web and mobile	Photos are read as bytes instead of file paths, which works on every platform

11. Limitations and Future Work

Limitation	Future work
`@aluedu.com` staff accounts skip email verification, so ownership of the address is not proven	Require verification for staff too, or approve staff in an admin screen
No push notifications when a status changes	Firebase Cloud Messaging triggered by a Cloud Function
One photo per report, fixed after submission, capped at 700 KB	Move to Cloud Storage if the project upgrades to Blaze
All reports load in one stream	Pagination or per-campus queries as the number of reports grows
Tests run locally, not on every push	Run the app and rules tests automatically with GitHub Actions
No staff comments on a report	A comment thread so staff can ask for details or explain a delay

12. Conclusion and Lessons Learned

The app meets its objectives: ALU users can report, browse, filter and track campus incidents, staff can resolve them, and every change appears live across screens, with access enforced on the server.

Lessons learned

- **The server must enforce the rules.** Hiding a button in the UI is not security; every permission has to be enforced in Firestore Security Rules.
- **Separating layers pays off.** Because screens never call Firebase directly, moving photos from Storage to Firestore changed one service and the provider, not the screens.
- **Streams make sync simple.** Listening to Firestore was simpler and more reliable than refreshing data by hand after each action.
- **Design within platform limits.** Billing limits and document-size limits shaped real decisions such as compressing photos and keeping them in a separate collection.
- **Testable code needs injection.** Passing services into providers made it possible to write 102 app tests that run without Firebase.

[Add one or two lessons in your own words.]

13. References

Flutter. (n.d.). *Build a form with validation*. Retrieved November 10, 2026, from https://docs.flutter.dev/cookbook/forms/validation

Flutter. (n.d.). *Simple app state management*. Retrieved November 10, 2026, from https://docs.flutter.dev/data-and-backend/state-mgmt/simple

Flutter. (n.d.). *Testing Flutter apps*. Retrieved November 10, 2026, from https://docs.flutter.dev/testing/overview

Google. (n.d.). *Firebase Authentication*. Firebase. Retrieved November 10, 2026, from https://firebase.google.com/docs/auth

Google. (n.d.). *Get started with Cloud Firestore Security Rules*. Firebase. Retrieved November 10, 2026, from https://firebase.google.com/docs/firestore/security/get-started

Google. (n.d.). *Usage and limits* (Cloud Firestore). Firebase. Retrieved November 10, 2026, from https://firebase.google.com/docs/firestore/quotas

Rousselet, R. (n.d.). *provider* [Computer software]. pub.dev. Retrieved November 10, 2026, from https://pub.dev/packages/provider